Experiment 9

Aim: Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

Dataset Source:

Stock Market Dataset https://www.kaggle.com/datasets/jacksoncrow/stock-market-dataset

Dataset Description:

The dataset contains historical stock price data.

Features:

- **Date**
- **Open**: Opening price
- **High**: Highest price
- **Low**: Lowest price
- **Close**: Closing price
- **Volume**

Target Variable:

- Predict **Close price**

Dataset Characteristics:

- Time series data
- Daily stock prices
- Continuous numerical values

Mathematical Formulation of the Algorithm:

Recurrent Neural Network (RNN)

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

$$y_t = W_y h_t$$

Where:

- ht: hidden state
- xt: input
- W: weights

LSTM Equations

Forget Gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input Gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Cell State:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Algorithm Limitations:

- Requires large sequential data
- Training is slow
- Sensitive to scaling
- RNN suffers from vanishing gradient problem
- LSTM is computationally expensive

Methodology / Workflow:

Steps:

1. Load dataset

```
df = pd.read_csv('GOGL.csv')  # or your stock file
df.head()
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	1997-02-07	112.500	113.125	108.125	110.0000	13.785553	1745800.0
1	1997-02-10	110.000	110.000	106.250	107.1875	13.433074	324200.0
2	1997-02-11	106.875	109.375	106.875	108.1250	13.550564	156200.0
3	1997-02-12	108.750	108.750	106.250	106.8750	13.393909	99300.0
4	1997-02-13	106.250	106.875	106.250	106.2500	13.315585	149400.0

2. Sort by date

```
df = df.sort_values('Date')

data = df['Close'].values.reshape(-1,1)

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)
```

3. Normalize data

```
X = []
y = []

time_steps = 60

for i in range(time_steps, len(data_scaled)):
    X.append(data_scaled[i-time_steps:i])
    y.append(data_scaled[i])

X, y = np.array(X), np.array(y)
```

4. Create sequences (time steps)

```
for i in range(time_steps, len(data_scaled)):
    X.append(data_scaled[i-time_steps:i])
    y.append(data_scaled[i])
```

5. Split into training and testing

```
split = int(0.8 * len(X))

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

6. Build LSTM model

```
model = Sequential()

model.add(LSTM(50, return_sequences=True, input_shape=(X.shape[1],1)))
model.add(LSTM(50))
model.add(Dense(1))
```

7. Train model

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

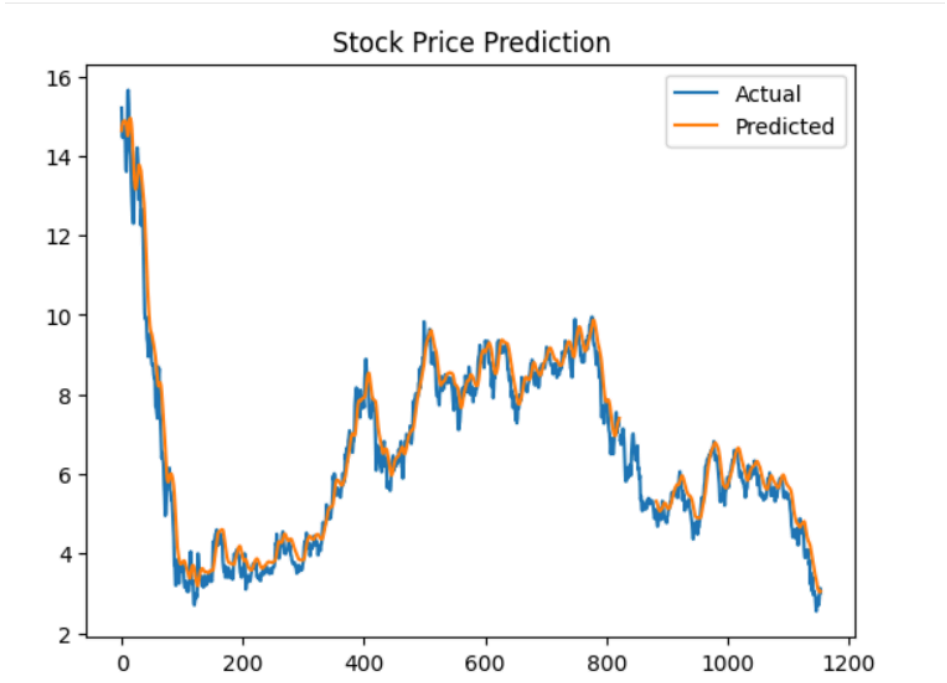
```
history = model.fit(X_train, y_train, epochs=10, batch_size=32)
```

8. Predict values

```
predictions = model.predict(X_test)
predictions = scaler.inverse_transform(predictions)
```

```
37/37 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step
```

9. Evaluate performance



Workflow Diagram:

Dataset → Preprocessing → Sequence Creation →

LSTM Model → Training → Prediction → Evaluation

Performance Analysis:

Metrics Used:

- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)

Observations:

- Model captures trends in stock prices
- Predictions follow actual data pattern

Interpretation:

- Lower RMSE → better prediction
- LSTM handles temporal dependencies effectively

Hyperparameter Tuning

Parameters Tuned:

- Number of LSTM layers
- Units (neurons)
- Batch size
- Epochs
- Time steps

Impact:

- More units → better learning (risk overfitting)
- More epochs → improved accuracy
- Time steps → better sequence understanding

SDG Goal	Implementation in Experiment
SDG 8: Decent Work and Economic Growth	Stock prediction models help investors make informed decisions, contributing to economic growth and financial stability.
SDG 9: Industry, Innovation and Infrastructure	Use of LSTM models promotes innovation in financial technology and predictive analytics systems.
SDG 4: Quality Education	Demonstrates practical implementation of deep learning models in time-series analysis.